



SDV • MODULE 35H • JAVA 25

Java Concurrency & Multithreading

Des Fondamentaux aux Threads Virtuels

Java 25 LTS

Project Loom

Spring Batch

WebFlux

9 Parties



Bonjour !



Je suis Séga SYLLA



Séga, c'est... le prénom



À PROPOS

Qui suis-je ?

 Basé à **Nantes**

 **CTO / Co-fondateur d'Ekwi@Tech** depuis 2025

 **Formateur Java / PHP** depuis 2015

 **Développeur professionnel** depuis 2010

 **Conception · Concurrency · Spring · CI/CD · Docker** au quotidien

TOUR DE TABLE

Et vous ?

- Quelques mots sur vous ?
- Qu'attendez-vous de cette formation ?

Objectifs du cours **Concurrence** (35h)

- Maîtriser les **fondamentaux** : processus, threads, ordonnancement, modèle mémoire JVM
- Écrire du code concurrent **correct** : synchronisation, atomicité, verrous, éviter les pièges
- Exploiter la concurrence **moderne** : ExecutorService, CompletableFuture, ForkJoin
- Adopter **Java 25 / Project Loom** : threads virtuels, concurrence structurée
- Construire des **pipelines production** : Spring Batch, WebFlux, haute performance

Ce que vous allez maîtriser

- **Fondamentaux de la concurrence** — processus, threads, ordonnancement, modèle JVM
- **Synchronisation** — conditions de compétition, interblocages, opérations atomiques, verrous
- **Concurrence Java moderne** — ExecutorService, CompletableFuture, ForkJoin
- **Java 25 Project Loom** — Threads virtuels, Concurrency structurée (stable)
- **Patterns production** — Spring Batch, WebFlux, pipelines haute performance

PARTIE 1

Fondamentaux

Processus, threads, ordonnancement, commutation de contexte, concurrence vs parallélisme — le socle avant tout code.

Processus

Threads

Ordonnancement

Modèle JVM

Amdahl

Qu'est-ce qu'un **Processus** ?

PROCESSUS (PID : 1042)

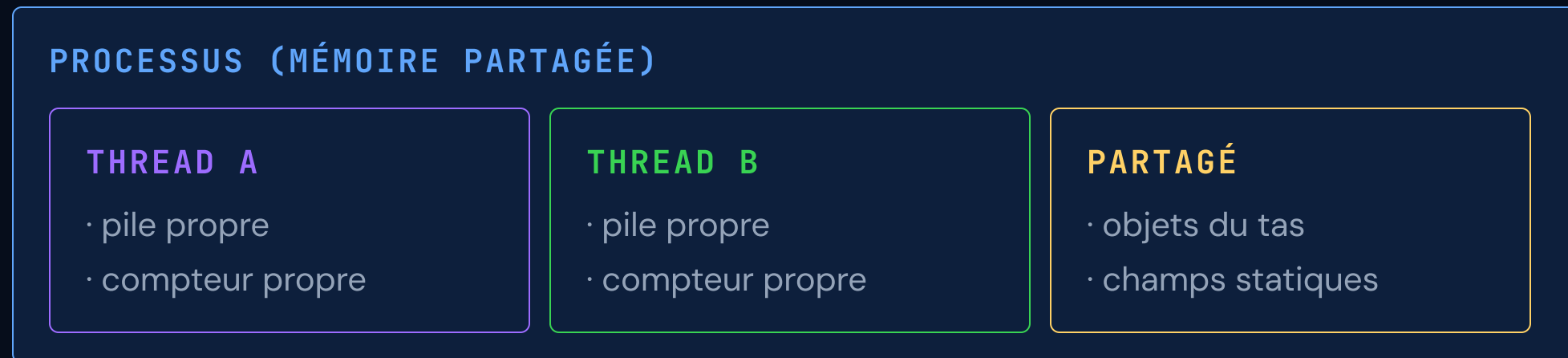
- Espace mémoire isolé (tas, pile, code)
- Descripteurs de fichiers & ressources OS
- Contexte d'exécution indépendant

Thread-1 (main)

Thread-2

- **Lourd** — l'OS alloue un espace d'adressage complet (~Mo de surcharge)
- **Isolé** — le crash d'un processus ne corrompt pas les autres
- **IPC requisite** pour la communication inter-processus (sockets, pipes)

Qu'est-ce qu'un Thread ?

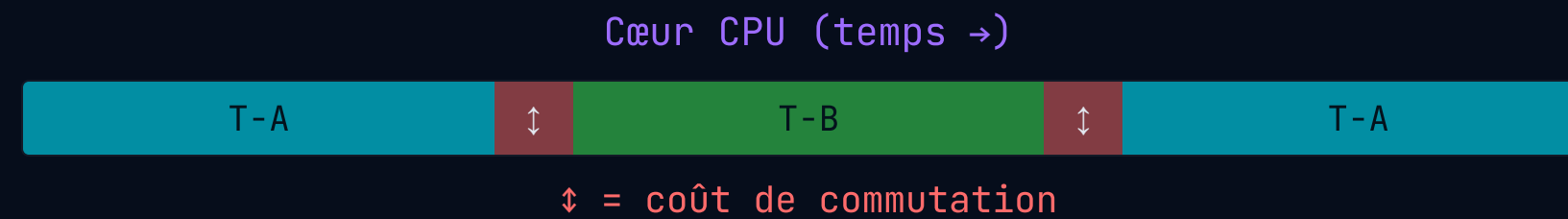


- **Léger** — partage la mémoire du processus, pile ~1 Mo par défaut
- **Risque** — état partagé sans coordination → corruption de données

Processus vs Thread

PROPRIÉTÉ	PROCESSUS	THREAD
Mémoire	Espace d'adressage propre	Partagée avec le processus
Coût de création	Élevé (ordre des ms)	Faible (ordre des µs)
Isolation des pannes	Isolation complète	Crash = tous les threads tués
Communication	IPC (sockets, pipes)	Mémoire partagée directe
Commutation contexte	Coûteuse (purge TLB)	Moins coûteuse (même AS)

Commutation de contexte



- **Sauvegarde & restauration** — registres CPU, PC, pointeur de pile → TCB noyau
- **~1-10 μs** par commutation ; invalidation TLB/cache pour les processus
- Trop de threads → thrashing : plus de temps à commuter qu'à calculer

Concurrence vs Parallélisme



- **Concurrence** — structure : gérer plusieurs choses à la fois (conception)
- **Parallélisme** — exécution : faire plusieurs choses à la fois (nécessite du matériel)

Pourquoi la concurrence est indispensable

- **Matériel moderne** — 8 à 128 cœurs ; code séquentiel = 1/128e de la capacité
- **L'I/O domine** — appels DB/réseau bloquants pendant des ms ; les threads servent d'autres requêtes
- **Attentes utilisateur** — latence <100ms à grande échelle = traitement concurrent obligatoire
- **Pipelines de données** — des millions d'enregistrements nécessitent de l'exécution parallèle
- **Économies** — meilleure utilisation CPU = moins de serveurs = coût infra réduit

Problème concret : 10 millions d'enregistrements

JAVA

```
// Naïve approach – sequential, single thread
for (Record r : records) { // 10,000,000 records
    validate(r); // ~0.5 μs CPU
    transform(r); // ~1 μs CPU
    insertToDB(r); // ~2 ms I/O wait ← BLOCKING
}

// Total ≈ 10M × 2ms = ~5.5 hours
// CPU ≈ 0.1% – 99.9% waiting on DB
// With 100 threads: ~3.3 minutes – ×100 gain, same hardware
```

Le CPU est inactif pendant l'attente BD – la concurrence comble ce vide.

CPU-bound vs I/O-bound

PROPRIÉTÉ	CPU-BOUND	I/O-BOUND
Goulot d'étranglement	Cycles CPU	Réseau / Disque / BD
Exemples	Chiffrement, ML, traitement image	Requêtes HTTP, SQL, lectures fichier
Nb threads optimal	≈ nombre de cœurs	Peut dépasser ×10 à ×1000
Stratégie Java 25	Threads platform + ForkJoin	Threads virtuels (Loom)

Loi d'Amdahl

$$S(n) = 1 / [(1 - p) + p/n]$$

S = accélération max | p = fraction parallélisable | n = nombre de processeurs

- Si 95% du code est parallélisable → accélération max = ×20 (même avec ∞ cœurs)
- La **portion séquentielle** est le plafond absolu — identifiez-la et éliminez-la
- Vrais goulots : verrous, contention I/O, pool de connexions BD
- Mesurez d'abord — n'optimisez jamais ce que vous n'avez pas profilé

Ce qui peut **mal tourner**

Condition de compétition

- › Deux threads lisent-modifient-écrivent simultanément → résultat corrompu

Interblocage (Deadlock)

- › Thread A attend B ; Thread B attend A → système gelé indéfiniment

Famine (Starvation)

- › Les threads prioritaires monopolisent les ressources ; les autres n'avancent jamais

Livelock

- › Les threads réagissent l'un à l'autre en boucle mais ne progressent pas

Partie 1 terminée. Suite → le modèle de threads Java et la création de threads.

PARTIE 2

Multithreading en Java

Du modèle de threads JVM à la création pratique, le cycle de vie, les threads démons et l'interruption coopérative.

Modèle JVM

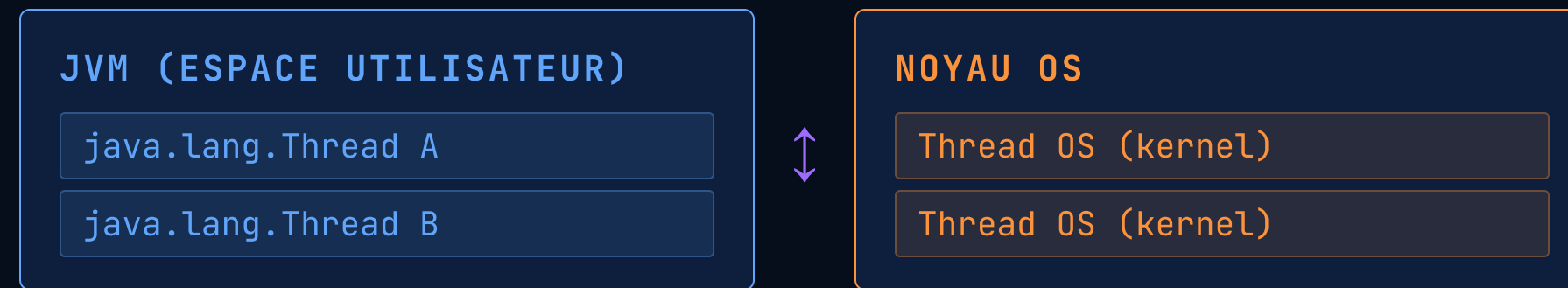
Runnable

Cycle de vie

Thread démon

Interruption

Modèle de threads JVM



- **Correspondance 1:1** — chaque thread Java platform = un thread OS (modèle héritage)
- Le noyau OS ordonnance les threads sur les cœurs CPU (préemptif)
- Pile par défaut : **512 Ko–1 Mo** → limite le nombre total à ~quelques milliers
- Java 25 Threads Virtuels cassent cette contrainte 1:1 (Partie 5)

Approche 1 : `extends Thread`

JAVA

```
public class MyTask extends Thread {  
  
    @Override  
    public void run() {  
        System.out.println(  
            "Running in: " + Thread.currentThread().getName()  
        );  
    }  
}  
  
MyTask t = new MyTask();  
t.start(); // ← start(), NEVER call run() directly!
```

- Java est à héritage unique — étendre `Thread` gaspille ce slot
- Mélange logique métier et mécanisme de threading — viole le SRP
- Préférer `Runnable` ou `Callable` en production

Approche 2 : implements Runnable

JAVA

```
// Lambda style – recommended (Java 8+)
Runnable task = () -> processRecord(r);

// Thread wraps the Runnable
Thread t = new Thread(task, "worker-1");
t.start();

// Inline – common for simple cases
new Thread(() -> processRecord(r)).start();

// Callable – like Runnable but returns a value
Callable<String> job = () -> fetchData(url);
```

- **Sépare** la définition de la tâche des mécanismes de threading
- Compatible avec `ExecutorService.submit()`

Cycle de vie d'un Thread



ÉTAT	QUAND	SORTIE
NEW	Créé, pas encore démarré	Appel start()
RUNNABLE	En cours ou prêt à s'exécuter	I/O, verrou, wait()
BLOCKED	Attend d'entrer dans synchronized	Verrou acquis
WAITING	wait() / join() / park()	notify() / fin join
TERMINATED	run() terminé ou exception	—

Thread démon vs Thread utilisateur

JAVA

```
// User thread – JVM waits for it before shutting down
Thread user = new Thread(() -> processOrders());
user.start();

// Daemon thread – killed abruptly on JVM shutdown (no cleanup!)
Thread daemon = new Thread(() -> {
    while (true) { collectMetrics(); sleep(1000); }
});
daemon.setDaemon(true); // MUST be set before start()
daemon.start();
```

Ne jamais utiliser un thread démon pour des opérations critiques – écritures BD, vidages de fichiers.

Interruption coopérative

JAVA

```
Thread worker = new Thread(() -> {
    while (!Thread.currentThread().isInterrupted()) {
        try {
            doWork();
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt(); // restore the flag!
            break;
        }
    }
    cleanup(); // always executed
});
worker.interrupt(); // cooperative signal, not forced
```

Avaler `InterruptedException` sans restaurer le drapeau est un bug silencieux très difficile à déboguer.

Piège : `run()` vs `start()`

INCORRECT – aucun thread créé

```
Thread t = new Thread(task);  
t.run(); // runs on THIS thread  
// No concurrency!
```

CORRECT – new OS thread

```
Thread t = new Thread(task);  
t.start(); // new OS thread  
// JVM calls run() on it
```

- `start()` demande à la JVM d'allouer un thread OS et d'appeler `run()` dessus
- Appeler `run()` directement = simple appel de méthode, zéro concurrence

Points clés Partie 2

- Préférer `Runnable` / lambda — sépare la tâche du mécanisme de threading
- Toujours appeler `start()`, jamais `run()` directement
- Restaurer le drapeau d'interruption lors de la capture d'`InterruptedException`
- Les threads démons sont tués à l'arrêt JVM — pas pour du travail critique
- La gestion brute des threads est source d'erreurs → Partie 4 : `ExecutorService`

PARTIE 3

Synchronisation & Pièges

Conditions de compétition, interblocages, opérations atomiques, verrous — le terrain le plus dangereux de la programmation concurrente.

Compétition

synchronized

volatile

Atomic

Interblocage

ReentrantLock

Condition de compétition — Le problème

JAVA

```
public class UnsafeCounter {  
    private int counter = 0; // shared mutable state  
  
    public void increment() {  
        counter++; // NOT atomic: ILOAD → IINC → ISTORE  
    }  
}  
  
// 100 threads × 1000 increments each  
// Expected: 100,000 – Actual: 87,432 (varies each run!)
```

- `counter++` compile en 3 instructions bytecode JVM — non atomique
- L'ordonnanceur peut préempter entre deux instructions → résultat non déterministe

Le mot-clé `synchronized`

JAVA

```
public class SafeCounter {  
    private int counter = 0;  
  
    // Method-level – lock on 'this'  
    public synchronized void increment() {  
        counter++;  
    }  
  
    // Block-level – finer granularity, preferred  
    public void incrementBlock() {  
        synchronized(this) { counter++; }  
    }  
}
```

- Un seul thread tient le moniteur — les autres passent **BLOCKED**
- Garantit **exclusion mutuelle** + **visibilité mémoire** (happens-before)
- Le verrou au niveau bloc minimise la contention — à préférer

Le mot-clé **volatile**

JAVA

```
public class Worker {  
    // Without volatile: thread may cache → stale reads  
    private volatile boolean running = true;  
  
    public void execute() {  
        while (running) { process(); } // always reads fresh value  
    }  
  
    public void stop() {  
        running = false; // immediately visible to all threads  
    }  
}
```

- **volatile** = **visibilité uniquement** — écritures vidées en mémoire principale
- **Pas atomique** — lecture-modification-écriture reste non sûre ; utiliser Atomic

Classes **Atomic** — sans verrou

JAVA

```
import java.util.concurrent.atomic.*;

AtomicInteger counter = new AtomicInteger(0);
counter.incrementAndGet();    // atomic ++, returns new value
counter.addAndGet(5);         // atomic += 5

// Compare-And-Swap (CAS) – hardware instruction
boolean ok = counter.compareAndSet(10, 20);
// Sets to 20 ONLY if current value == 10

// High contention: LongAdder outperforms AtomicLong
LongAdder adder = new LongAdder();
adder.increment(); adder.sum();
```

- Sans verrou — utilise l'instruction CAS du CPU ; aucun thread bloqué
- Plus rapide que `synchronized` sous contention faible à modérée

Interblocage — Attente circulaire

THREAD A

- ✓ détient Verrou 1
- ⌚ attend Verrou 2

DEADLOCK
X

THREAD B

- ✓ détient Verrou 2
- ⌚ attend Verrou 1

JAVA

```
// Thread A: lockA → lockB
synchronized(lockA) { synchronized(lockB) { doWork(); } }
// Thread B: lockB → lockA ← DEADLOCK!
synchronized(lockB) { synchronized(lockA) { doWork(); } }
```

Solution : toujours acquérir les verrous dans le MÊME ordre global dans TOUS les threads.

Livelock & Famine

PROBLÈME	SYMPTÔME	SOLUTION
Interblocage	Threads gelés en BLOCKED définitivement	Ordre des verrous, tryLock avec timeout
Livelock	CPU élevé, threads actifs, aucun progrès	Backoff aléatoire, règles de priorité
Famine	Thread RUNNABLE qui ne s'exécute jamais	Verrous équitables, files bornées

- ReentrantLock(true) — mode équitable : threads servis en FIFO (plus lent mais juste)
- Détecter avec : jstack, VisualVM, async-profiler

ReentrantLock — Utilisation avancée

JAVA

```
ReentrantLock lock = new ReentrantLock(); // or (true) for fair mode

// tryLock – non-blocking attempt with timeout
if (lock.tryLock(500, TimeUnit.MILLISECONDS)) {
    try {
        criticalSection();
    } finally {
        lock.unlock(); // ALWAYS in finally!
    }
} else {
    handleTimeout(); // graceful degradation
}
```

- `unlock()` dans `finally` obligatoire — verrou non libéré = blocage permanent
- Supporte `Condition` pour wait/signal plus fin qu'avec `synchronized`

ReadWriteLock — Lecture/Écriture

JAVA

```
ReadWriteLock rw = new ReentrantReadWriteLock();

// Multiple concurrent readers allowed
public Data read() {
    rw.readLock().lock();
    try { return cache; } finally { rw.readLock().unlock(); }
}

// Exclusive writer – blocks all readers and writers
public void write(Data d) {
    rw.writeLock().lock();
    try { cache = d; } finally { rw.writeLock().unlock(); }
}
```

- Idéal pour les **charges à dominante lecture** — caches, configuration, tables de référence

Collections thread-safe

CLASSE	USAGE	STRATÉGIE
ConcurrentHashMap	Map lectures/écritures concurrentes	CAS + verrouillage par segment
CopyOnWriteArrayList	Beaucoup de lectures, rares écritures	Copie à chaque écriture
LinkedBlockingQueue	Producteur-consommateur	Bornée, bloque si pleine/vide
ConcurrentLinkedQueue	File sans verrou	Basée sur CAS, non bloquante

Ne jamais utiliser Collections.synchronizedList() en haute concurrence – verrou grossier qui détruit le débit.

Arbre de décision **synchronisation**

BESOIN	OUTIL	POURQUOI
Compteur simple	AtomicInteger	Sans verrou, rapide
Drapeau d'arrêt	volatile boolean	Visibilité seule, pas de verrou
Section critique	bloc synchronized	Simple, fiable
Timeout / tryLock	ReentrantLock	Contrôle avancé
Cache dominé lecture	ReadWriteLock	Max concurrence lecture
Producteur-consommateur	BlockingQueue	Coordination intégrée

PARTIE 4

Concurrence Java Moderne

ExecutorService, pools de threads, Future, CompletableFuture, ForkJoin, streams
parallèles — la boîte à outils de production.

ExecutorService

ThreadPoolExecutor

CompletableFuture

ForkJoin

Streams parallèles

Pourquoi **new Thread()** ne suffit pas

- **Pas de réutilisation** — créer/détruire des threads est coûteux ; les pools amortissent ce coût
- **Pas de contre-pression** — création illimitée → OutOfMemoryError sous charge
- **Pas de gestion du cycle de vie** — arrêt, attente de complétion, gestion d'erreurs difficiles
- **Pas de valeur de retour** — Runnable ne peut pas retourner de résultat ni lever d'exception vérifiée
- `ExecutorService` résout tout cela avec une API propre et éprouvée

ExecutorService — Bases

JAVA

```
ExecutorService executor = Executors.newFixedThreadPool(8);

// Submit a task – returns a Future
Future<String> future = executor.submit(() -> fetchData(url));
executor.submit(() -> processRecord(r)); // fire-and-forget

// Graceful shutdown – mandatory!
executor.shutdown();
executor.awaitTermination(60, TimeUnit.SECONDS);
```

- 8 threads réutilisés pour toutes les tâches soumises — pas d'explosion
- Les tâches s'accumulent dans la file quand tous les threads sont occupés
- Toujours appeler `shutdown()` — fuite d'executor empêche l'arrêt de la JVM

Types de pools — choisir judicieusement

FABRIQUE	COMPORTEMENT	QUAND L'UTILISER
<code>newFixedThreadPool(n)</code>	n threads, file illimitée	CPU-bound, charge prévisible
<code>newCachedThreadPool()</code>	Grandit/rétrécit à la demande	Tâches courtes, charge variable
<code>newSingleThreadExecutor()</code>	1 thread, exécution ordonnée	File de tâches sérialisée
<code>newScheduledThreadPool(n)</code>	Tâches planifiées/périodiques	Jobs récurrents type cron
<code>newVirtualThreadPerTaskExecutor()</code>	Thread virtuel par tâche	I/O-bound, Java 25

`newCachedThreadPool()` sous charge I/O intensive peut créer des milliers de threads et provoquer un OOM.

ThreadPoolExecutor — Paramétrage fin

JAVA

```
ThreadPoolExecutor pool = new ThreadPoolExecutor(  
    4,                                // corePoolSize  
    8,                                // maximumPoolSize  
    60L, TimeUnit.SECONDS,            // keepAliveTime  
    new LinkedBlockingQueue<>(1000), // file bornée  
    new ThreadPoolExecutor.CallerRunsPolicy() // politique de rejet  
);  
// Politiques : Abort | CallerRuns | Discard | DiscardOldest
```

- **File bornée** est critique — évite l'épuisement mémoire sous charge
- `CallerRunsPolicy` = contre-pression naturelle : ralentit le producteur

Future & Callable

JAVA

```
Callable<User> job = () -> fetchFromDB(userId);
Future<User> future = executor.submit(job);

// ... do other work while task executes ...

User u = future.get(5, TimeUnit.SECONDS); // blocking

// Non-blocking check
if (future.isDone()) { User u = future.get(); }
future.cancel(true); // interrupt if running
```

- `Future.get()` est bloquant — limite la composabilité et le débit
- Solution : `CompletableFuture` — non bloquant et composable

CompletableFuture — Chaînage

JAVA

```
CompletableFuture<Report> cf =  
    CompletableFuture.supplyAsync(() -> fetchUser(id))  
        .thenApply(u -> enrich(u))           // sync transformation  
        .thenCompose(u -> fetchOrders(u))    // async flatMap  
        .thenApply(orders -> buildReport(orders));  
  
// Retrieve result (block only when necessary)  
Report r = cf.join();
```

- `supplyAsync` tourne sur le `ForkJoinPool` commun par défaut — passer un `executor` explicite en prod
- `thenApply` = `map` | `thenCompose` = `flatMap` (étapes asynchrones)

CompletableFuture — Combinaison

JAVA

```
// Fan-out: 3 tasks in parallel, wait for all
CompletableFuture<Void> all = CompletableFuture.allOf(
    processChunk(chunk1),
    processChunk(chunk2),
    processChunk(chunk3)
);
all.join(); // wait for all three

// First to complete wins
CompletableFuture.anyOf(mirror1, mirror2).join();

// Combine two results
cf1.thenCombine(cf2, (a, b) -> merge(a, b));
```

- `allOf` — attendre toutes les tâches (pattern de traitement par lots)
- `anyOf` — le premier gagne (requêtes en miroir, géo-redondance)

CF — Gestion des erreurs

JAVA

```
CompletableFuture<String> cf = fetchData()
    .exceptionally(ex -> {
        log.error("Failed", ex);
        return defaultValue; // fallback value
    })
    .handle((result, ex) -> {
        if (ex != null) return fallback();
        return result; // always executed
    })
    .whenComplete((r, ex) -> metrics.record(r, ex));
```

- `exceptionally` — récupérer d'une erreur avec une valeur de repli
- `handle` — s'exécute toujours (comme `finally`) ; inspecte résultat et erreur

ForkJoinPool & vol de tâches

JAVA

```
class SumTask extends RecursiveTask<Long> {  
    protected Long compute() {  
        if (size <= THRESHOLD) return sequentialSum();  
        SumTask left = new SumTask(leftHalf);  
        left.fork(); // async  
        return new SumTask(rightHalf).compute() + left.join();  
    }  
}  
  
long res = ForkJoinPool.commonPool().invoke(new SumTask(...));
```

- **Vol de tâches** — threads inactifs volent les tâches des threads occupés
- Optimal pour les algorithmes **diviser-pour-régner** CPU-bound
- Utilisé en interne par les **streams parallèles**

Streams parallèles

JAVA

```
// Sequential – single thread
records.stream()
    .filter(Record::isValid)
    .map(this::transform)
    .collect(Collectors.toList());

// Parallel – uses ForkJoinPool.commonPool()
records.parallelStream()
    .filter(Record::isValid)
    .map(this::transform)
    .collect(Collectors.toList());
```

- API simple — mais utilise le `commonPool` partagé qui affecte toute la JVM
- **Jamais** pour du travail I/O-bound ou quand l'ordre compte
- Mesurer — le parallèle est plus lent sur les petits jeux de données (surcharge)

Points clés **Partie 4**

BESOIN	OUTIL
Pool de threads géré	<code>ExecutorService</code> / <code>ThreadPoolExecutor</code>
Async avec valeur de retour	<code>Callable</code> + <code>Future</code>
Pipeline asynchrone composable	<code>CompletableFuture</code>
Diviser-pour-régner CPU	<code>ForkJoinPool</code> + <code>RecursiveTask</code>
Transformation de données simple	<code>parallelStream()</code>

Toujours passer un executor explicite à `CompletableFuture.supplyAsync()` – ne jamais dépendre du `commonPool` en production.

PARTIE 5

Threads Virtuels

Java 25

Project Loom révolutionne la concurrence — des millions de threads légers, sans callbacks, sans complexité réactive.

Project Loom

Threads porteurs

Concurrence structurée

Épinglage

Les threads platform ne **passent pas à l'échelle**

THREAD PLATFORM

- ~1 Mo de pile (fixe)
- Correspondance 1:1 thread OS
- Création coûteuse
- Max ~10 000 threads/JVM



I/O BLOQUANT

- Thread attend, inactif
- Thread OS épinglé
- Cœur porteur gaspillé
- Plafond de débit atteint

- 10 000 threads × 1 Mo = **10 Go rien que pour les piles**
- La plupart des threads sont inactifs en attente d'I/O — ressources gaspillées

Threads Virtuels — Création

JAVA

```
// Java 25 – Thread.ofVirtual()
Thread vt = Thread.ofVirtual().start(() -> {
    System.out.println("Virtual: " + Thread.currentThread());
});

// Named virtual thread
Thread named = Thread.ofVirtual()
    .name("ingestion-", 0)
    .start(() -> processRecord(r));

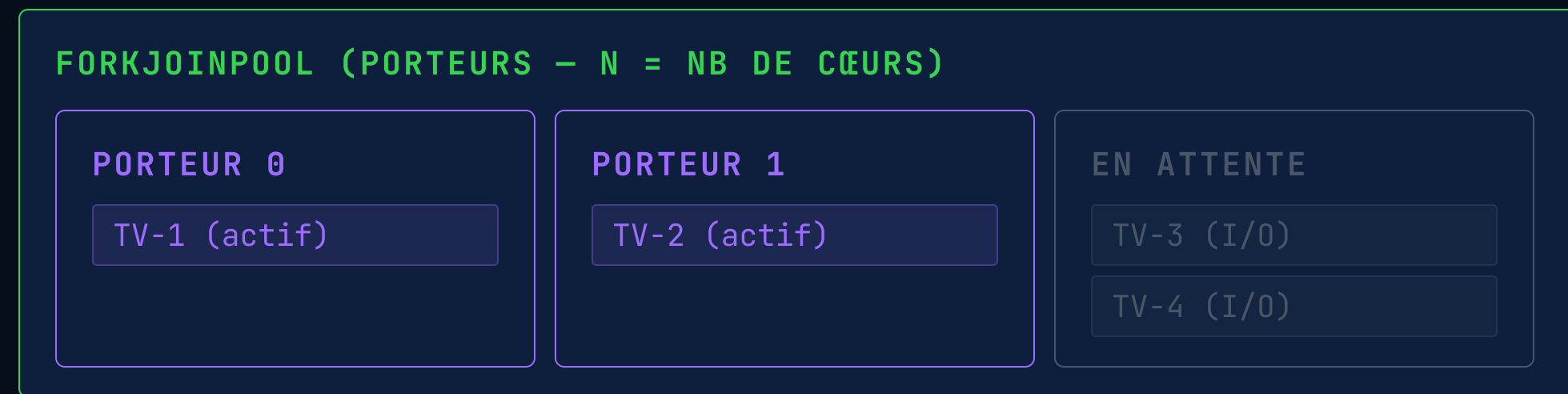
// Executor – one virtual thread per task
ExecutorService vExec =
    Executors.newVirtualThreadPerTaskExecutor();
```

- API identique aux threads platform — migration sans friction
- Des milliers à la milliseconde — créez-en des millions sans OOM

Threads Virtuels vs Platform

PROPRIÉTÉ	THREAD PLATFORM	THREAD VIRTUEL
Taille de pile	~1 Mo (fixe)	~1 Ko (élastique, grandit)
Thread OS	1:1	M:N (plusieurs TV → peu de porteurs)
Coût de création	~1 ms, appel OS	~1 µs, JVM uniquement
Nombre max pratique	~10 000	Des millions
I/O bloquant	Épingle le thread OS	Démonte, porteur libéré
Idéal pour	CPU-bound	I/O-bound (BD, HTTP, fichiers)

Montage / Démontage



- TV rencontre un I/O bloquant → **démontage** du porteur, état sauvegardé dans le tas
- Le porteur est immédiatement libéré pour exécuter un autre TV
- Quand l'I/O se termine → TV remonté sur n'importe quel porteur disponible

Concurrence Structurée

JAVA

```
// Java 25 – stable API (JEP 505, finalisé en Java 24 JEP 499)
try (var scope = new StructuredTaskScope.ShutdownOnFailure()) {
    Future<User>    u = scope.fork(() -> fetchUser(id));
    Future<Order>  o = scope.fork(() -> fetchOrder(id));

    scope.join().throwIfFailed(); // wait and throw if any failed
    return new Response(u.get(), o.get());
} // scope closes – cancels any unfinished subtask
```

- **Portée de durée de vie** — les sous-tâches ne survivent pas à leur scope (pas de fuites)
- `ShutdownOnFailure` — annuler tout si l'un échoue
- `ShutdownOnSuccess` — annuler tout dès que le premier réussit

ScopedValue — **replace ThreadLocal**

JAVA

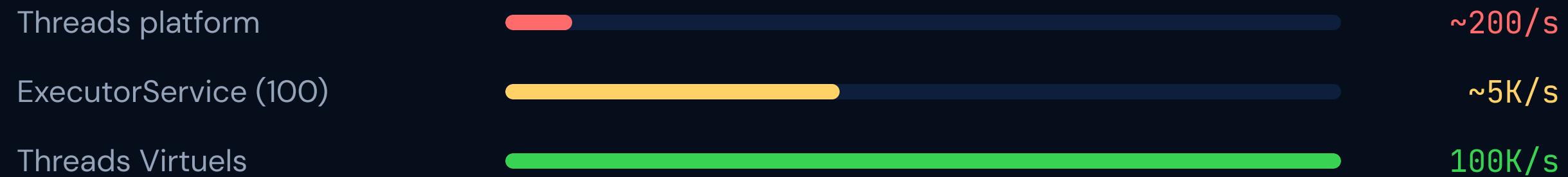
```
// ThreadLocal – risk of leaks with virtual threads
ThreadLocal<String> ctx = ThreadLocal.withInitial(() -> "");

// ScopedValue – stable API (JEP 506, Java 25) – immutable, no leak
ScopedValue<RequestContext> CTX = ScopedValue.newInstance();

ScopedValue.where(CTX, new RequestContext(traceId))
    .run(() -> {
        handleRequest(); // CTX.get() accessible here
    });
// After run() – CTX is automatically unbound
```

- `ThreadLocal` avec TV : pas de pool → pas de nettoyage → risque de fuite mémoire
- `ScopedValue` est **immuable** et **automatiquement nettoyé**

Comparaison de débit



- Scénario I/O-bound : chaque tâche effectue un appel HTTP de 10 ms
- Les threads virtuelsaturent la capacité I/O, non le nombre de threads
- Résultat : amélioration **×50** du débit par rapport à un pool fixe de 100

Épinglage **résolu en Java 24** (JEP 491)

Java 21-23 – épinglait le porteur

```
// Java 21-23: synchronized + blocking I/O
// pinned the carrier thread – avoid!
synchronized(lock) {
    Thread.sleep(1000);
    // carrier was blocked
}
```

Java 25 – synchronized OK avec VT

```
// Java 25: JEP 491 fixed pinning –
// synchronized unmounts correctly
synchronized(lock) {
    Thread.sleep(1000); // OK now
} // ReentrantLock still preferred
```

- **JEP 491 (Java 24)** — `synchronized` ne bloque plus le porteur : démontage correct
- `ReentrantLock` reste recommandé pour le contrôle fin (`tryLock`, `fairness`, `Condition`)
- Détecter les épinglages résiduels : `-Djdk.tracePinnedThreads=full`

Points clés **Partie 5**

- Threads Virtuels = threading M:N sur la JVM — des millions de threads légers possibles
- L'I/O bloquant démonte le TV de son porteur — le porteur sert d'autres TV
- Utiliser `Executors.newVirtualThreadPerTaskExecutor()` pour les services I/O-bound
- **JEP 491 (Java 24)** — `synchronized` ne pêne plus les porteurs ; `ReentrantLock` reste préféré
- Concurrency Structurée + `ScopedValue` = APIs stables, code concurrent plus sûr et lisible

PARTIE 6

Patterns de Traitement Parallèle

Producteur-consommateur, fan-out/fan-in, traitement par lots, primitives de coordination — construire des pipelines haute performance.

Producteur-Consommateur

Traitement par lots

CountDownLatch

Sémaphore

Pipeline

Pattern Producteur-Consommateur

```
JAVA
BlockingQueue<Record> queue = new LinkedBlockingQueue<>(1000);

// Producer – reads source, publishes to queue
executor.submit(() -> {
    while (hasData()) queue.put(readNext()); // blocks if full
});

// Consumers × N – parallel processing
for (int i = 0; i < 8; i++) {
    executor.submit(() -> {
        while (running) process(queue.take()); // blocks if empty
    });
}
```

- **File bornée** = contre-pression naturelle — le producteur ralentit
- `put()` bloque si pleine ; `take()` bloque si vide — pas d'attente active

Producteur-Consommateur — Schéma



- Régler : vitesse producteur, taille file, nb consommateurs, taille de lot
- Surveiller : profondeur de file = indicateur avancé du retard consommateur
- Pattern poison pill : envoyer une sentinelle `null` ou objet spécial pour signaler la fin

Traitement par **lots parallèles**

JAVA

```
// Partition records into batches
List<List<Record>> batches = partition(records, 500);

// Process all batches concurrently
List<CompletableFuture<Void>> futures = batches.stream()
    .map(batch -> CompletableFuture
        .runAsync(() -> insertBatch(batch), executor))
    .collect(Collectors.toList());

CompletableFuture.allOf(futures.toArray(new CompletableFuture[0]))
    .join(); // wait for all inserts
```

- ▶ Insert JDBC par lot : 500 lignes en un seul statement → **x10 à x50 plus rapide**
- ▶ Régler la taille du lot : plus grand = moins d'allers-retours ; trop grand = pression mémoire

Pattern Pipeline



- Chaque étape s'exécute en concurrence — les enregistrements coulent comme une chaîne de montage
- Étapes connectées par `BlockingQueue` — découplées, tampons indépendants
- L'étape goulot détermine le débit global (Loi de Little)
- Scaler : répliquer l'étape lente horizontalement (plusieurs consommateurs par file)

CountDownLatch — Coordination

```
JAVA
CountDownLatch latch = new CountDownLatch(workers.size());

workers.forEach(w -> executor.submit(() -> {
    try {
        w.process(); // do the work
    } finally {
        latch.countDown(); // signal completion (always)
    }
}));

latch.await(60, TimeUnit.SECONDS); // wait for all workers
mergeResults(); // safe: all done
```

- Usage unique : le compteur atteint zéro → tous les threads en attente sont libérés
- `countDown()` dans `finally` — évite que le verrou ne soit jamais libéré

Sémaphore — Limitation de débit

JAVA

```
// Max 10 concurrent DB connections
Semaphore semaphore = new Semaphore(10);

// Virtual threads + semaphore = controlled concurrency
try (var exec = Executors.newVirtualThreadPerTaskExecutor()) {
    records.forEach(r -> exec.submit(() -> {
        semaphore.acquire();           // wait for a permit
        try { insert(r); }
        finally { semaphore.release(); } // always release!
    }));
}
```

- Pattern classique : threads virtuels illimités + Sémaphore = usage BD borné
- Toujours `release()` dans `finally` — permis perdu = famine permanente

Optimisation du **débit**

- **Profiler d'abord** — identifier l'étape goulot avant de régler les comptages de threads
- **Insertions BD par lot** — `PreparedStatement.addBatch()` + `executeBatch()`
- **Dimensionner les files** — trop petite = famine ; trop grande = pression GC
- **Éviter la contention de verrous** — partitionner les données par clé
- **Surveiller** — profondeur file, threads actifs, pauses GC, attente connexion BD

PARTIE 7

Traitement Réactif

I/O non bloquant, boucle d'événements, contre-pression, Manifeste Réactif — et quand choisir le réactif plutôt que les threads virtuels.

I/O non bloquant

Boucle d'événements

Contre-pression

Éditeur-Abonné

I/O Bloquant vs Non-bloquant

BLOQUANT (UN THREAD PAR REQUÊTE)

- Thread émet l'appel I/O
- Thread attend (bloqué)
- I/O terminé → thread reprend
- 1 requête = 1 thread = 1 Mo de pile

VS

NON-BLOQUANT (BOUCLE D'ÉVÉNEMENTS)

- Émettre I/O → enregistrer callback
- Thread traite d'autres événements
- I/O terminé → callback invoqué
- N requêtes = 1 thread = boucle

- Non-bloquant : 1 thread gère des milliers d'opérations I/O en vol simultanément

Manifeste **Réactif**

Disponible

- › Répond dans des délais bornés, même sous charge

Résilient

- › Reste disponible en cas de panne — isolation, réplication

Élastique

- › Scale à la hausse ou à la baisse selon la demande

Orienté messages

- › Échange asynchrone — couplage faible, transparence de localisation

Contre-pression — Contrôle de flux

Éditeur
rapide

tampon PLEIN → perte / OOM

Abonné lent

Éditeur
contrôlé

request(n) → contre-pression

Abonné en
contrôle

- L'abonné signale sa demande : `request(n)` — l'éditeur envoie au plus n éléments
- Évite qu'un producteur rapide ne submerge un consommateur lent — pas d'OOM

Éditeur → Abonné

JAVA

```
// Reactive Streams API (java.util.concurrent.Flow since Java 9)
Publisher<Data> publisher = createPublisher();
publisher.subscribe(new Subscriber<Data>() {
    public void onSubscribe(Subscription s) {
        s.request(100); // request 100 items (backpressure)
    }
    public void onNext(Data d)      { process(d); }
    public void onError(Throwable t) { handle(t); }
    public void onComplete()        { finish(); }
});
```

- API bas niveau — utiliser Project Reactor (Flux/Mono) ou RxJava en pratique
- Spring WebFlux est construit sur Project Reactor (Partie 8)

Réactif **vs** Threads Virtuels

SCÉNARIO	RÉACTIF (WEBFLUX)	THREADS VIRTUELS
Flux de données en continu	Excellent ✓	Possible
Haute concurrence I/O	Excellent ✓	Excellent ✓
Services CRUD simples	Surdimensionné ✗	Idéal ✓
Courbe d'apprentissage	Élevée ✗	Faible ✓
Libs bloquantes (JDBC)	Nécessite R2DBC ✗	Fonctionne nativement ✓
Pipelines riches en opérateurs	Écosystème riche ✓	Codage manuel

Par défaut, choisir les Threads Virtuels en Java 25. Choisir le réactif pour le streaming ou si la stack est déjà réactive.

PARTIE 8

Spring Batch & WebFlux

Des frameworks industriels qui appliquent les patterns appris — jobs parallèles, HTTP réactif, accès BD non bloquant.

Spring Batch

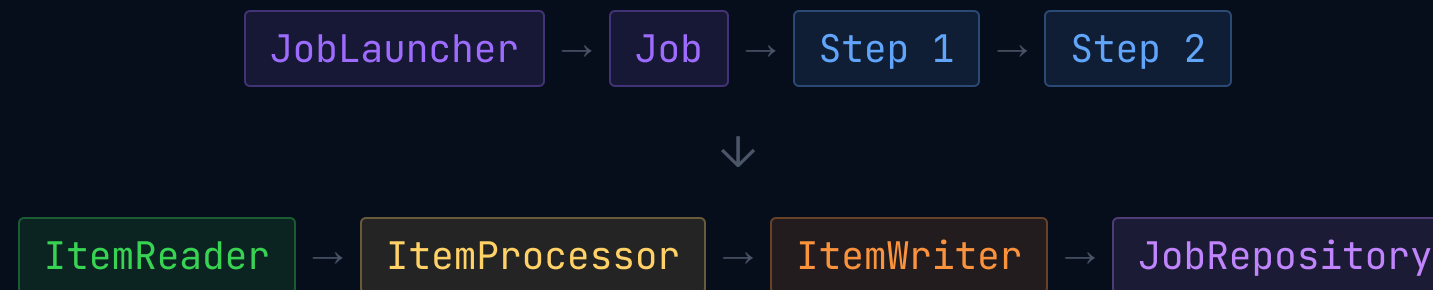
Step partitionné

WebFlux

Flux / Mono

R2DBC

Architecture **Spring Batch**



- **Traitement orienté chunks** — lire N éléments, traiter, écrire dans une transaction
- **JobRepository** — persiste l'état d'exécution ; jobs redémarrables sans surcoût
- Retry, skip, listeners et métadonnées d'exécution intégrés

Spring Batch — Steps Parallèles

JAVA

```
@Bean
public Job importJob(Step stepA, Step stepB) {
    Flow parallelFlow = new FlowBuilder<SimpleFlow>("parallelFlow")
        .split(new SimpleAsyncTaskExecutor())
        .add(flow(stepA), flow(stepB)) // run concurrently
        .build();
    return jobBuilder.start(parallelFlow).end().build();
}
```

- `split(executor)` — les steps s'exécutent en concurrence sur l'executor fourni
- Utiliser pour les sources indépendantes : step CSV + step API + step BD simultanément

Spring Batch — Step Partitionné

JAVA

```
@Bean
public Step partitionedStep(Step workerStep) {
    return stepBuilder.partitioner("worker", partitioner())
        .step(workerStep)           // step executed per partition
        .taskExecutor(taskExecutor())
        .gridSize(8)                // 8 parallel partitions
        .build();
}
// Partitioner: splits 0-10M into 8 ranges (0-1.25M, 1.25M-2.5M...)
```

- Chaque travailleur lit sa propre partition indépendamment
- gridSize = nombre de travailleurs parallèles — caler sur la taille du pool de connexions BD

Spring **WebFlux** — HTTP Réactif

JAVA

```
// Annotation style – reactive return types
@GetMapping("/users")
public Flux<User> getAllUsers() {
    return repository.findAll(); // non-blocking stream
}

@PostMapping("/ingest")
public Mono<Report> ingest(@RequestBody Mono<Data> data) {
    return data.flatMap(this::validate)
               .flatMap(this::processAsync);
}
```

- `Mono<T>` = 0 ou 1 élément ; `Flux<T>` = 0 à N éléments (flux continu)
- S'exécute sur la boucle d'événements Netty — pas de thread par requête

R2DBC — Base de données réactive

JAVA

```
// Reactive Spring Data repository
@Repository
public interface UserRepository
    extends ReactiveCrudRepository<User, Long> {

    Flux<User> findByStatus(String status);
    Mono<Long> countByDepartment(String dept);
}

// Service usage (non-blocking end to end)
return repository.findByStatus("ACTIVE")
    .filter(User::isEligible)
    .flatMap(this::enrichAsync)
    .collectList();
```

- R2DBC : pilote BD réactif non bloquant (PostgreSQL, MySQL, H2)
- Nécessaire pour maintenir une stack entièrement non bloquante

Comparaison des **approches**

APPROCHE	COMPLEXITÉ	DÉBIT	IDÉAL POUR
Threads bruts	Élevée	Moyen	Apprentissage uniquement
ExecutorService	Moyenne	Bon	CPU + I/O mixte
Threads Virtuels	Faible	Élevé	Services I/O-bound
Spring Batch	Moyenne	Élevé	Pipelines de données batch
WebFlux + R2DBC	Élevée	Très élevé	Streaming, APIs réactives

Points clés **Partie 8**

- **Spring Batch** — jobs batch industriels ; steps partitionnés = traitement parallèle de données
- **WebFlux** — couche HTTP réactive ; idéal pour le streaming, SSE, APIs non bloquantes
- **R2DBC** — nécessaire pour maintenir le non-blocage de bout en bout dans une stack réactive
- Ni Spring Batch ni WebFlux ne nécessitent une connaissance approfondie du framework pour démarrer

PARTIE 9

Projet Final

Appliquez tout ce que vous avez appris. Construisez une plateforme d'ingestion haute performance — et affrontez-vous sur le classement.

Ingestion de données

Pipeline concurrent

Benchmark

Classement

Plateforme d'Ingestion Haute Performance

- **Scénario** — traiter 5 millions d'enregistrements depuis CSV / JSON / API HTTP
- **Valider** — vérification de schéma, règles métier, dédoublonnage
- **Transformer** — enrichir, normaliser, calculer les champs dérivés
- **Persister** — insertion par lots dans PostgreSQL avec débit maximal
- **Mesurer** — enregistrements/seconde, mémoire utilisée, taux d'erreur, latence p99

Architecture du Pipeline



- Contrat imposé : implémenter l'interface `DataIngestionPipeline`
- Chaque étape indépendamment scalable par multiplication des consommateurs

Contrat de code **imposé**

JAVA

```
public interface DataIngestionPipeline {  
    IngestionReport ingest(Path dataFile, IngestionConfig config);  
}  
  
public record IngestionConfig(  
    int batchSize,           // e.g. 500  
    int parallelism,        // e.g. 8  
    ProcessingStrategy strategy // VT | EXECUTOR | BATCH | REACTIVE  
) {}  
  
public record IngestionReport(  
    long    recordsProcessed,  
    long    recordsFailed,  
    Duration duration,  
    double  throughputPerSecond  
) {}
```

Choisissez votre **stratégie**

Option A

Threads Virtuels + JDBC par lots

› `newVirtualThreadPerTaskExecutor + PreparedStatement.addBatch()`

Option B

ExecutorService + CompletableFuture

› Pool fixe + pipeline CF + fan-in allOf + insert par lots

Option C

Spring Batch Partitionné

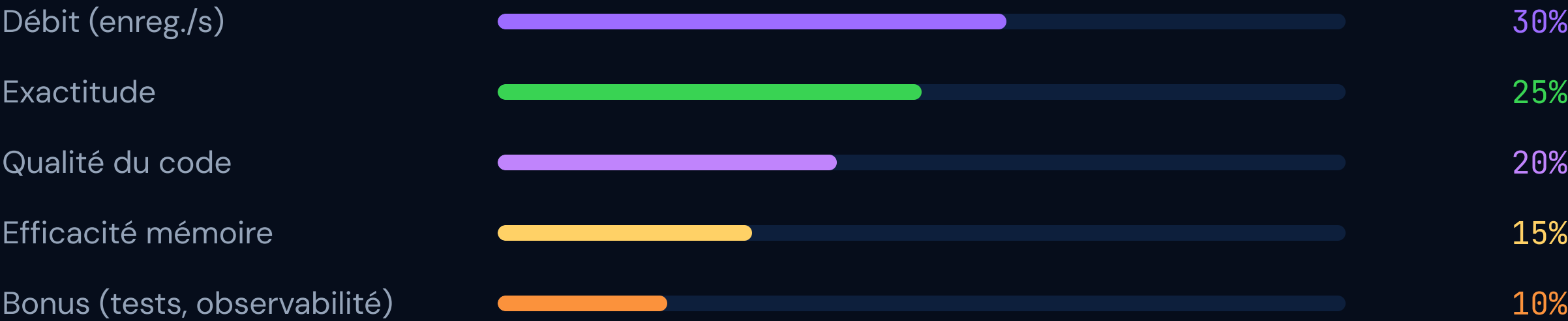
› `Reader/Processor/Writer + step partitionné + gridSize=8`

Option D ★ Avancé

Spring WebFlux + R2DBC

› Pipeline Flux + `flatMap(concurrency)` + insert BD réactif

Critères de notation



Classement de la **promotion**

1er	equipe_alpha	247 832 enreg./s	Option A – Threads Virtuels
2e	batch_heroes	183 410 enreg./s	Option C – Spring Batch
3e	reactive_squad	171 290 enreg./s	Option D – WebFlux
4	cf_masters	134 500 enreg./s	Option B – ExecutorService

Résultats soumis à un endpoint partagé. Classements publiés en direct lors du jour de démo.

Ce qu'il faut **rendre**

- ☐ Code source dans un dépôt Git (GitHub / GitLab)
- ☐ Interface DataIngestionPipeline implémentée
- ☐ README : schéma d'architecture + justification du choix de stratégie
- ☐ Résultats de benchmark : débit, mémoire, latence — avec capture du profileur
- ☐ Démo de 5 minutes : exécution en direct + explication des choix de concurrence
- ☐ Bonus : tests unitaires des composants concurrents (jcstress ou personnalisé)

COURS TERMINÉ

Codez vite. Pensez concurrent.

Java Concurrency & Multithreading · Sup de Vinci

9 Parties

Java 25 LTS

Project Loom

35 Heures

Séga SYLLA – Formateur

 [in/séga-sylla-5000804b](https://www.linkedin.com/company/séga-sylla-5000804b)  [sega.sylla](https://discord.com/invite/sega.sylla)

